Lab 2 Tasks: The Noisy Channel

UART, Interrupts, and Modbus RTU

Contents

Overview 1

Objectives 2

Required Hardware and Software 2

Pre-Laboratory Exercises 3
- Part "A": Direct Register Access . . . 3
- Part "B": The HAL . . . 3

Part "A": Direct Register Access 4
- **Task A1: Send text and random digits** . . . 6
- **Task A2: Measure and decode a UART character** . . . 7
- **Task A3: Keep reception working during a busy-wait delay** . . . 8

Part "B": The HAL 10
- **Task B1: Control LD2 through Modbus** . . . 12
- **Task B2: Read registers and handle invalid requests** . . . 14
- **Task B3: Compare software and hardware CRC calculation** . . . 16

Demonstration and Sign-off 18

Appendix
- A The Serial Monitor . . . 19
- B The Modbus master program . . . 20

Overview

The development board becomes a receiver and transmitter of messages on a noisy serial line (Figure 1):

- Your computer is the *master*. It sends *requests* to the board: switch the user LED LD2, or send the values of the counters. The requests obey Modbus RTU, a standard protocol of industrial devices.
- Noise changes some bits. The last two bytes of each message hold a cyclic redundancy check (CRC). The CRC finds bit errors.
- Your program on the board is the *slave*. It performs supported requests and sends replies. It ignores messages with a CRC mismatch.
- With no reply, your computer sends the request again.

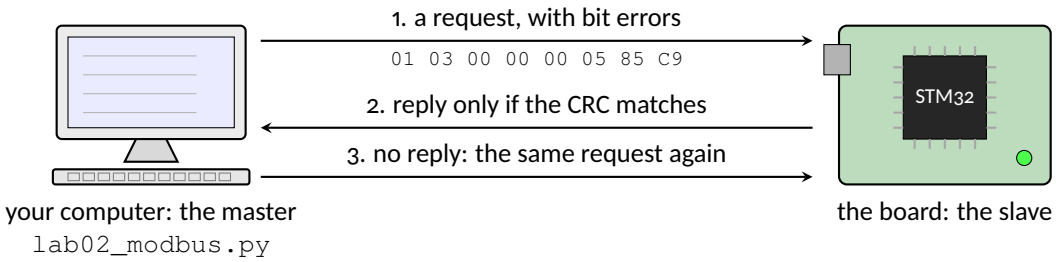


Figure 1: The noisy channel. The board ignores messages with a CRC mismatch.

Structure of this laboratory

This laboratory has two parts.

- **Part "A", direct register access.** Send a random digit to your computer. Send back each character that you type. Measure the characters on the oscilloscope.
- **Part "B", the hardware abstraction layer (HAL).** Make the board a Modbus RTU slave with the HAL functions from STMicroelectronics (ST). Calculate the CRC with a hardware unit of the chip.

Objectives

- To configure a universal asynchronous receiver-transmitter (UART), a serial interface, through its registers, and to set its baud rate, the number of bits per second.
- To find the bits of a UART character, and the bit time, on the oscilloscope.
- To connect a peripheral to a pin through an alternate function.
- To demonstrate that a busy loop loses received characters, and to prevent this loss with an interrupt and its handler.
- To receive the messages of a standard protocol, Modbus RTU, by interrupt with the HAL.
- To detect CRC mismatches and reply only when the received CRC matches the calculated CRC.
- To calculate a CRC with a hardware unit of the chip, and to compare its time with the time in software.

Required Hardware and Software

- The NUCLEO-G491RE development board.
- An oscilloscope.
- Two male-to-male jumper wires: one for ground, and one for the probed signal.
- The Serial Monitor extension of VS Code, from Microsoft (Appendix A).
- Python 3, with the `pyserial` package.
- The program `lab02_modbus.py`, from the course repository, for Part "B".

Pre-Laboratory Exercises

Do the pre-laboratory exercises before the laboratory session. Each part has its own exercises. Before the exercises of a part, read its chapters in the booklet. The contents of the booklet list the chapters of each part.

Part "A": Direct Register Access

RM0440 describes `CR1` and `ISR` two times. Read the description for FIFO mode disabled, the reset state: its title ends in "[alternate]".

1. **Find the registers of LPUART1.** Use RM0440 to give:
   (a) the bit numbers of `UE`, `RE`, `TE`, and `RXNEIE` in `CR1` (RM0440 section 41.7.2, p. 1807).
   (b) the bit numbers of `TXE`, `RXNE`, and `ORE` in `ISR`, for FIFO mode disabled (RM0440 section 41.7.8, p. 1820).
   (c) the register and the bit that enable the clock of LPUART1 (RM0440 section 7.4.18, p. 311).

2. **Calculate the baud rates.** Select one baud rate for LPUART1 and USART1. Their kernel clock is 170 MHz.
   (a) Calculate `BRR` for LPUART1, rounded to an integer. Make sure that it obeys the limits in Table 4 of the booklet.
   (b) Calculate `BRR` for USART1, rounded to an integer.
   (c) Calculate the bit time, and the time of one frame of 10 bits.

3. **Find the alternate functions.** In the alternate function table (DS13122 section 4.10, p. 63), find the AF number of:
   (a) `LPUART1_TX` on PA2 and `LPUART1_RX` on PA3.
   (b) `USART1_TX` on PC4.

4. **Find the interrupt.**
   (a) In the vector table (RM0440 section 14.3, p. 437), find the number of the LPUART1 interrupt, in the column *Position*. In `startup_stm32g491xx.s`, find the name of its handler.
   (b) Give the `NVIC_ISER` register and the bit that enable this interrupt.

Part "B": The HAL

5. **Make a time budget.** The line runs at 115200 baud, 8N1: each byte takes 10 bits. Calculate:
   (a) the time of one byte, and of a request of 8 bytes.
   (b) the idle time of 3.5 character times at the end of a message.
   (c) the time of the reply to a read of 5 registers: 15 bytes.

6. **Read a Modbus message.** A master sends these 8 bytes, in hexadecimal: `01 06 00 00 00 01 48 0A`. Use Appendix A of the booklet.
   (a) Give the address, the Modbus function code, the register, and the value.
   (b) Give the CRC as a 16-bit number.
   (c) Noise changes the last byte to `0x0B`. Give the reply of the slave. Explain it.

Part "A": Direct Register Access

In Part "A", configure the peripherals through direct register access. Do not call `HAL_*` functions in the code you add. Your program (Figure 2):

- sends the line `Lab 2 ready.` at reset.
- sends a random digit, `0` to `9`, at each press of button B1.
- sends back each character that you type in the Serial Monitor. This return is an *echo*.

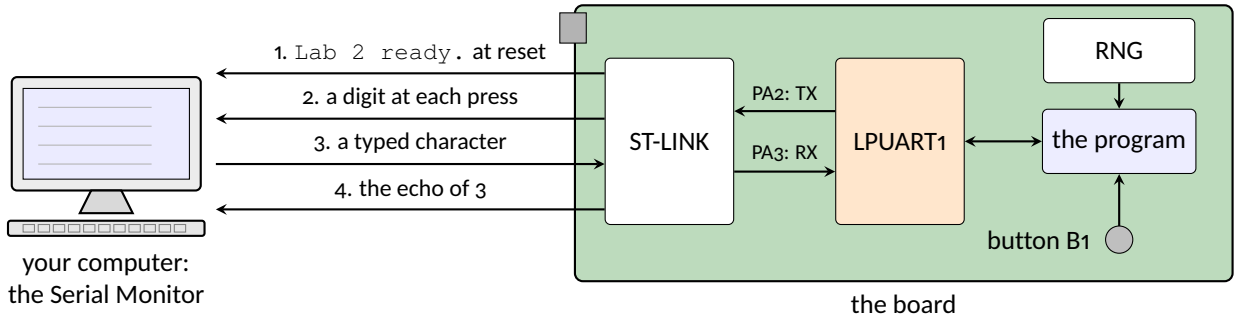


Figure 2: The program of the board in Part "A", and the path of its characters.

Create a new project as in Step 2 of Lab 0. Keep the generated initialization code and its 170 MHz clock configuration. Complete Task A1, then extend its program for Task A2, and then Task A3.

Use one baud rate for LPUART1, USART1, and the Serial Monitor, with 8 data bits, no parity, and 1 stop bit. In the Serial Monitor, set the line ending to **None** (Appendix A). After each change, build the project and flash the board.

The preparation

Copy Listings 1 and 2 into `main.c`. Set `LPUART1_BRR_VALUE` from Exercise 2(a).

In `USER CODE BEGIN 2`, enable the clocks of ports *A* and *C*. Configure PC13 as an input and PA5 (LD2) as an output.

Listing 1 supplies the random number generator (RNG) functions.

```
/* RNG: 32-bit random numbers from the true random number generator */
static void rng_init(void)
{
  RCC->CRRCR |= RCC_CRRCR_HSI48ON;            /* start HSI48 */
  while ((RCC->CRRCR & RCC_CRRCR_HSI48RDY) == 0U) { }
  RCC->CCIPR &= ~RCC_CCIPR_CLK48SEL;          /* 00: 48 MHz = HSI48 */
  RCC->AHB2ENR |= RCC_AHB2ENR_RNGEN;          /* RNG clock on */
  RNG->CR |= RNG_CR_RNGEN;                    /* start the RNG */
}

static uint32_t rng_read(void)
{
  while ((RNG->SR & RNG_SR_DRDY) == 0U) {     /* wait for a number */
    if ((RNG->SR & RNG_SR_SEIS) != 0U) {      /* seed error */
      RNG->SR &= ~RNG_SR_SEIS;                /* clear SEIS */
      for (uint32_t i = 0; i < 12U; i++) {
        (void)RNG->DR;                        /* discard 12 words */
      }
    }
  }
  return RNG->DR;
}
```

Listing 1: Part “A”: The RNG functions. rng_init() starts the RNG, and rng_read() gives a 32-bit random number. Put them in USER CODE BEGIN 0.

```
/* USER CODE BEGIN PD */
#define LPUART1_BRR_VALUE   /* your value from Exercise 2 */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
static uint32_t previous = 0;   /* button B1 on the previous pass */
/* USER CODE END PV */

/* USER CODE BEGIN 0 */
/* rng_init() and rng_read() */

static void uart_init(void)          /* LPUART1: your baud, 8N1 */
{
}

static void uart_putc(char character) /* sends one character */
{
}

static void uart_puts(const char *text) /* sends a string */
{
}
/* USER CODE END 0 */
```

Listing 2: Part “A”: The parts that you complete. Each comment gives the section.

Task A1: Send text and random digits

In this task, the board sends text to your computer. You write the three UART functions of Listing 2, and the characters travel from LPUART1 through the ST-LINK and USB to the Serial Monitor. At reset, the board sends the line `Lab 2 ready.` At each press of button B1, it sends one random digit, `0` to `9`.

What to write.

- `uart_init()`, in this order:
  (a) Enable the clock of LPUART1 (Exercise 1(c)).
  (b) Put PA2 and PA3 in alternate function mode, with the AF number of Exercise 3(a) (Section 2.6 of the booklet). Keep the configuration of the other pins of port A: PA13 and PA14 connect the debugger.
  (c) Write `LPUART1_BRR_VALUE` to `BRR`.
  (d) Set `TE`, `RE`, and `UE` of `CR1` (Exercise 1(a)).
- `uart_putc(character)`: wait until `TXE` of `ISR` is 1 (Exercise 1(b)). Then write `character` to `TDR`.
- `uart_puts(text)`: send each character of the string `text` with `uart_putc()`.
- `USER CODE BEGIN 2:` call `uart_init()` and `rng_init()`. Then send `"Lab 2 ready.\r\n"`.
- The `while (1)` loop: at each press of button B1, send one digit as a character, `'0'` to `'9'`; `rng_read()` of Listing 1 gives a 32-bit random number. While button B1 is held down, send only one digit: `previous` of Listing 2 holds the state of button B1 in the previous pass.

What to demonstrate and explain.

- **Demonstrate the startup message and digits.**
  (a) Open the Serial Monitor at your baud rate. Press the black reset button B2. Make sure that the startup line appears.
  (b) Press button B1 ten times. Each press must produce one digit.
  (c) Hold button B1 down. Only one digit must appear during the press.
- **Demonstrate the pin configuration.** Pause the program. In the debugger's **WATCH** section, add:
  (a) `(GPIOA->MODER >> 4) & 0xF`: the mode fields of PA2 and PA3.
  (b) `(GPIOA->AFR[0] >> 8) & 0xFF`: their alternate function fields.
  In each result, PA2 occupies the lower half and PA3 the upper half.
- **Explain the pin configuration.** Compare these fields with your `uart_init()`. Explain how they connect PA2 and PA3 to LPUART1.

Task A2: Measure and decode a UART character

In this task, you see the characters of Task A1 as an electrical signal. Transmit each character to the computer and on PC4 for oscilloscope observation (Figure 3). The pins of LPUART1 reach only the ST-LINK, so a copy of each character goes out on USART1, whose TX pin, PC4, is on a header. On the oscilloscope, you then measure the frame of one character and decode its bits, and compare them with what the Serial Monitor shows.

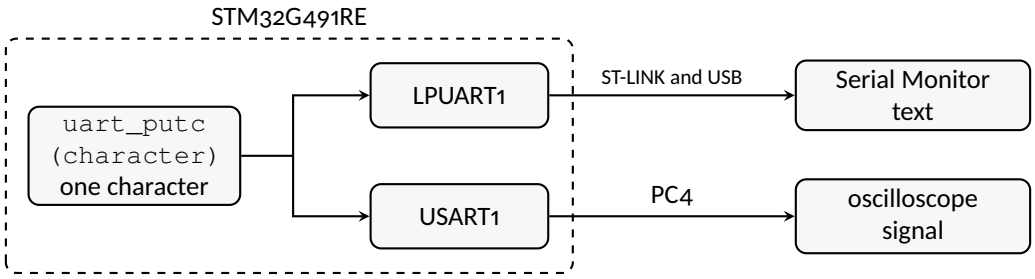


Figure 3: Task A2: Two paths for the same character.

What to write.

- `USER CODE BEGIN PD`: add `#define USART1_BRR_VALUE`, with your `BRR` value for USART1 (Exercise 2(b)).
- `uart_init()`: at its end, configure USART1 in the same way as LPUART1, with four differences:
  - Its clock enable bit is in `RCC->APB2ENR`; the device header gives its name.
  - Its TX pin is PC4, with the AF number of Exercise 3(b).
  - Write `USART1_BRR_VALUE` to `USART1->BRR`.
  - It only transmits: `RE` stays 0.
- `uart_putc(character)`: also send `character` on USART1. Wait until `TXE` of USART1 is 1, then write `character` to the `TDR` of USART1.

What to demonstrate and explain.

- **Demonstrate the transmitted text.** Make sure that the Serial Monitor displays the same text as in Task A1.
- **Demonstrate one frame.** Probe PC4. Press button B1 once. Capture the digit's frame and keep the image.
- **Measure the timing.** The stop bit and idle line are both high. Bit 7 of a digit is 0, so the start of its stop bit has a rising edge.
  (a) Measure one bit time. Compare it with your prediction from Exercise 2(c).
  (b) Measure from the start bit's falling edge to the stop bit's rising edge. Compare this interval with nine predicted bit times.
  (c) Calculate the baud rate from your measured bit time.
- **Decode the character.** Read the 8 data bits, starting with bit 0 after the start bit. Give the character in hexadecimal. Compare it with the digit in the Serial Monitor.

Task A3: Keep reception working during a busy-wait delay

A program cannot always wait for the UART: it also has other work, for example a calculation. Characters arrive also while the program does this work. In this task, you find out what happens to them, and how an interrupt prevents their loss.

The busy-wait delay stands for this work: a loop that only counts to 2 000 000 (Listing 3). While it counts, the program checks nothing else. LD2 toggles before each delay.

The three stages (Figure 4). Test each stage before you start the next. *What to write* and *What to demonstrate and explain* below are in the same three groups.

- **(a) Echo by polling.** The main loop checks LPUART1 in every pass, and sends back each character that arrives.
- **(b) Add the delay.** The loop now checks LPUART1 only between two delays. Some characters are lost: you find out which ones, and why.
- **(c) Echo by interrupt.** When a character arrives, LPUART1 interrupts the delay, and the handler sends the character back. Then the delay continues, and no character is lost.

The flag `ECHO_BY_INTERRUPT` selects (b) or (c) in one `main.c`, so that you can demonstrate both at the test.

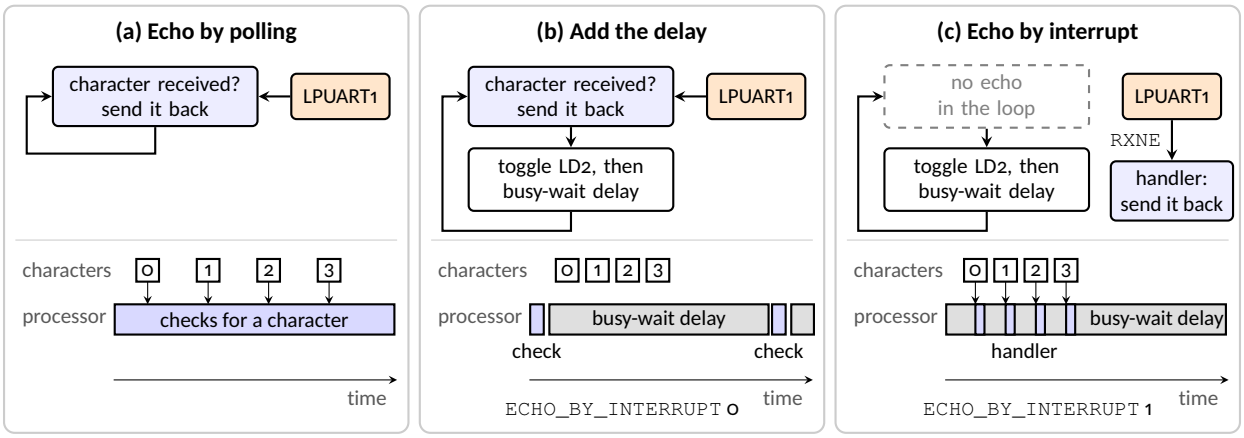


Figure 4: Task A3: The three stages. Top: the main loop. Bottom: what happens in time, not to scale. The flag selects (b) or (c).

```
/* USER CODE BEGIN PD: add */
#define WORK_PASSES       2000000UL   /* delay loop iterations */
#define ECHO_BY_INTERRUPT        0    /* 0: polling, 1: interrupt */

    /* while (1): add after the echo, before USER CODE END WHILE */
    /* busy-wait delay: toggle LD2, then wait */
    GPIOA->ODR ^= GPIO_ODR_OD5;
    for (volatile uint32_t i = 0; i < WORK_PASSES; i++) { }
```

Listing 3: Task A3: The busy-wait delay and the flag that selects the echo. Each comment gives its location in main.c.

What to write.

- **(a) Echo by polling.**
  - In the `while (1)` loop, send back each character that LPUART1 receives, with `uart_putc()` (Section 2.9 of the booklet).
- **(b) Add the delay.**
  - Add Listing 3 at the places that its comments give.
- **(c) Echo by interrupt.**
  - The handler: its name is your answer to Exercise 4(a). It echoes each received character, and clears `ORE` after an overrun (Section 3.1 of the booklet). Do not make it `static`: the default handler of the startup file, an endless loop, would then stay in place.
  - The interrupt: set `RXNEIE` in LPUART1 (Exercise 1(a)) and its bit in the NVIC (Exercise 4(b)). These are the enable items of Table 6 of the booklet.
  - The flag: `ECHO_BY_INTERRUPT` of Listing 3 selects the echo with `#if`. At 0, the loop echoes, and the interrupt stays off. At 1, the interrupt is on, and the loop does not echo. Only the echo changes: the delay and the code for button B1 stay in the loop.

What to demonstrate and explain.

- **(a) Echo by polling.**
  - **Demonstrate the echo.** Type `0123456789` in the Serial Monitor's message box. Press Enter to send the ten characters without pauses. Each character must return.
- **(b) Add the delay.**
  - **Observe what returns.** Send `0123456789` two times. Observe what returns each time.
  - **Measure the delay.** Probe PA5. Measure the time between successive edges and keep the image. Each interval is one loop pass, spent almost entirely on the delay.
  - **Demonstrate the reception flags.** After sending the two lines, pause the program. In **WATCH**, add expressions for `RXNE` and `ORE`, each giving 0 or 1. Use Task A1's method and the bit numbers from Exercise 1(b). Do not read `RDR`: this clears `RXNE`.
  - **Explain the lost characters.** Explain both replies using `RXNE`, `ORE`, your frame time from Exercise 2(c), and the measured delay duration.
- **(c) Echo by interrupt.**
  - **Demonstrate both modes.** Set `ECHO_BY_INTERRUPT` to 1. Build and flash the board. Send `0123456789` while LD2 blinks: every character must return. Then set the flag to 0, build and flash again, and send the same line.
  - **Explain the difference.** Why does the interrupt version receive every character while the delay runs?

Part “B”: The HAL

The board is a Modbus RTU slave with the address 1 (Figure 5). The master program is `lab02_modbus.py`. Set up the master program with Appendix B. After each change, build the project and flash the board.

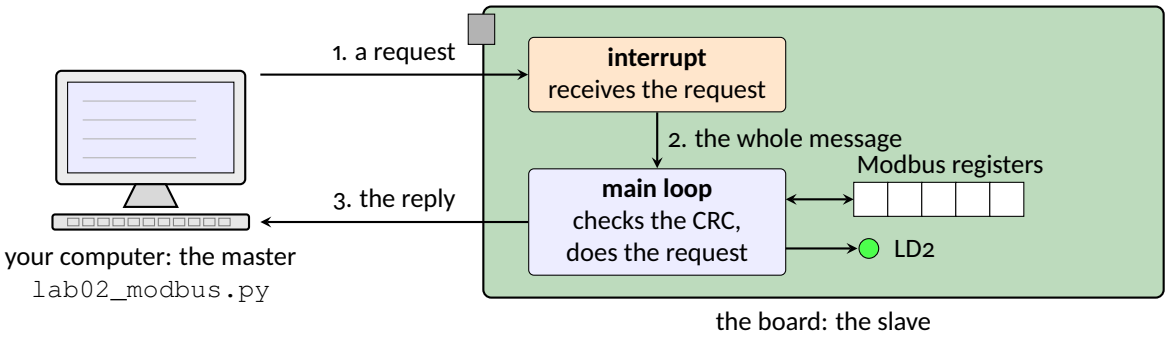


Figure 5: The program of the board in Part “B”. The interrupt receives each request, and the main loop handles it.

The project

Copy `lab02_partB.ioc` and `lab02_modbus.py` from the `lab02-uart` folder of the course repository into the project folder. Complete Task B1, then extend its program for Task B2, and then Task B3.

The declarations

The three tasks use the global declarations of Listing 4. Copy them into `main.c`. Each marker gives the section. The array `regs` holds the five Modbus registers of the board (Table 1).
A CRC matches when the received CRC equals the CRC calculated from the preceding message bytes. Both message counters start at zero after a board reset. They count messages of at least 4 bytes; each retransmission is another received message.

Name in the code	Content	Access
`REG_LD2`	LD2: 1 on, 0 off.	read and write
`REG_GOOD`	Messages addressed to this board with a matching CRC, including requests that receive exception replies.	read
`REG_BAD`	Received messages with a CRC mismatch, regardless of their address.	read
`REG_CRC_SW`	The clock cycles of the CRC in software (Task B3).	read
`REG_CRC_HW`	The clock cycles of the CRC in the CRC unit (Task B3).	read

Table 1: The Modbus registers of the board. The master program calls them `LD2`, `good`, `bad`, `crc_sw`, and `crc_hw`.

```c
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define MB_ADDRESS      1U   /* the Modbus address of the board */
#define MESSAGE_MAX    64U   /* bytes of one message, at most */
#define NUM_REGS        5U   /* holding registers 0 to 4 */
#define REG_LD2         0U   /* LD2: 1 on, 0 off */
#define REG_GOOD        1U   /* messages with a correct CRC */
#define REG_BAD         2U   /* messages with a wrong CRC */
#define REG_CRC_SW      3U   /* cycles of the CRC in software */
#define REG_CRC_HW      4U   /* cycles of the CRC in the CRC unit */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
static uint8_t  rx[MESSAGE_MAX];          /* the bytes that arrive */
static uint8_t  message[MESSAGE_MAX];     /* a complete message */
static volatile uint16_t message_len = 0; /* its bytes; 0: none */
static uint16_t regs[NUM_REGS];           /* the holding registers */
/* USER CODE END PV */
```

Listing 4: Part “B”: The declarations. Each marker gives the section.

The preparation

In `USER CODE BEGIN MX_GPIO_Init_2`, configure PA5 as an output, as in Task C1 of Lab 1. Include the `GPIO_InitTypeDef` variable and the HAL clock enable for port *A*.
In `USER CODE BEGIN 2`, start the cycle counter (Table 10 of the booklet). In the same section, give LPUART1 priority 1 (Figure 22 of the booklet), with the HAL function of Section 6.3 of the booklet.
Copy Listing 5 into `USER CODE BEGIN 0`.

```c
/* CRC-16/MODBUS of byte_count bytes. Polynomial 0xA001 (0x8005 with
   bits in reverse order), start value 0xFFFF. A message holds it low
   byte first. */
static uint16_t modbus_crc16(const uint8_t *data, uint32_t byte_count)
{
  uint16_t crc = 0xFFFFU;
  for (uint32_t i = 0; i < byte_count; i++) {
    crc ^= data[i];
    for (uint32_t b = 0; b < 8U; b++) {
      if ((crc & 1U) != 0U) {
        crc = (uint16_t)((crc >> 1) ^ 0xA001U);
      } else {
        crc = (uint16_t)(crc >> 1);
      }
    }
  }
  return crc;
}
```

Listing 5: Part “B”: The CRC of Modbus RTU. Put it in USER CODE BEGIN 0.

Task B1: Control LD2 through Modbus

In this task, the board becomes a Modbus slave. Modbus is a protocol of industrial devices, such as sensors, motor drives, and energy meters. The master program on your computer sends write requests to it (Table 11 of the booklet). The board updates LD2 and replies with a copy of the request (Figure 6). Like a slave on a shared line, it ignores a message with a CRC mismatch or another slave's address.

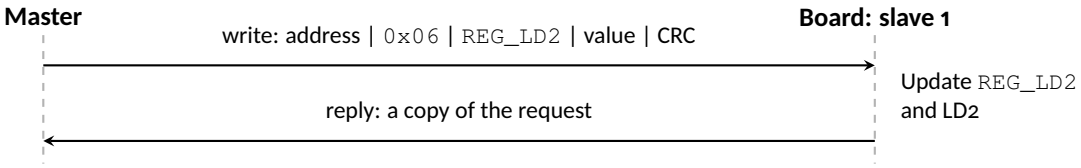


Figure 6: Task B1: A write request changes LD2. The reply repeats the request.

```
/* USER CODE BEGIN 0 */
/* modbus_crc16() of Listing 5 */

void HAL_UARTEx_RxEventCallback(UART_HandleTypeDef *huart,
                                uint16_t size)    /* line idle */
{
}

void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart)  /* error */
{
}

static int crc_ok(const uint8_t *message_bytes,
                  uint16_t message_length)        /* 1: CRC matches */
{
}

static void handle_message(const uint8_t *message_bytes,
                           uint16_t message_length)
{
}
/* USER CODE END 0 */
```

Listing 6: Task B1: The functions that you write, in their order in USER CODE BEGIN 0.

What to write. Define the four functions in `USER CODE BEGIN 0`, in the order of Listing 6. Define both callbacks without `static`, so they replace the HAL defaults (Section 6.2 of the booklet).

1. Receive a request and pass it to the loop. The HAL receives into `rx`; the callback copies a completed request into `message`. The variable `message_len` is its byte count: 0 means the buffer is free; a nonzero value means it is occupied (Listing 4 and Figure 21 of the booklet).

(a) **Start reception**, in `USER CODE BEGIN 2`. The HAL receives bytes only after a call to `HAL_UARTEx_ReceiveToIdle_IT()` (Table 8 of the booklet). Pass the handle `&hlpuart1`, the buffer `rx`, and its capacity, `MESSAGE_MAX`. The call returns immediately; the HAL fills `rx` and calls the reception callback when reception ends.

(b) **`HAL_UARTEx_RxEventCallback()`** passes the received bytes to the loop. The `size` bytes are in `rx` (Figure 19 of the booklet). If `message_len` is 0, copy them into `message`, then set `message_len` to `size`. Otherwise, discard the new bytes. In both cases, call the reception function again to receive the next request.

(c) **The `while (1)` loop** processes the saved request. When `message_len` is nonzero, call `handle_message(message, message_len)`. Clear `message_len` only after the function returns, to free the buffer.

(d) **`HAL_UART_ErrorCallback()`** restores reception after an error, such as an overrun. The HAL stops reception before calling it. Call the reception function again.

2. Validate the request and control LD2.

(a) **`crc_ok()`** checks the received CRC. Calculate the CRC of all bytes except the last two with `modbus_crc16()` (Listing 5). Return 1 if the last two bytes match it, in the byte order of Appendix A of the booklet; otherwise, return 0.

(b) **`handle_message()`** counts messages and handles writes. Implement its checks and actions in this order:

(i) If the message has fewer than 4 bytes, return without replying.

(ii) Use `crc_ok()` and the address byte to update the counters of Table 1: a CRC mismatch in `regs[REG_BAD]`, and a matching CRC for `MB_ADDRESS` in `regs[REG_GOOD]`. After a CRC mismatch or another slave's address, return without replying.

(iii) Accept only an 8-byte request with function `0x06` that writes `REG_LD2`, as in Exercise 6. For any other request, return without replying.

(iv) Switch LD2 off for value 0, and on for any other value. Store 0 or 1 in `regs[REG_LD2]`.

(v) Reply with a copy of the request using `HAL_UART_Transmit()`. Its last parameter is a time limit in milliseconds.

What to demonstrate and explain.

3. Test LED control.

(a) Click **Stop Monitoring** in the Serial Monitor. Only one program at a time can open the port.

(b) Run `lab02_modbus.py` with `--write-only`. Demonstrate that LD2 changes and each round reports `OK` (Appendix B).

(c) **Explain reception.** When do the callback and loop use `message`? What happens if it is busy? How does reception restart after an error?

Task B2: Read registers and handle invalid requests

In this task, you add Modbus function `0x03`, so that the master can also read the registers of the board. Writes keep the value policy of Task B1. A request that the board cannot do gets an exception reply: the master then knows that the board received the request but did not do it. A corrupted request gets no reply, and the master sends it again (Figure 7).

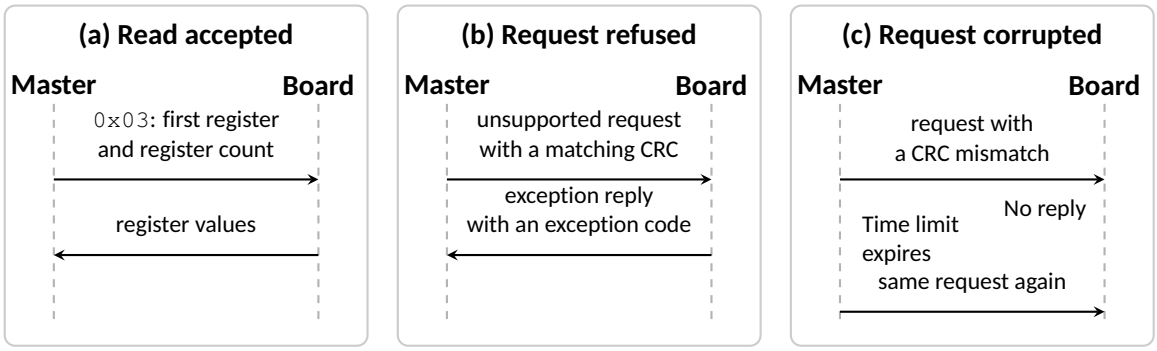


Figure 7: Task B2: Three separate exchanges with slave 1. Time runs downward in each panel.

```
static void reply(uint8_t *reply_bytes, uint16_t length_without_crc)
{
}

static void exception(uint8_t function, uint8_t code)
{
}
```

Listing 7: Task B2: The new functions. Put them in USER CODE BEGIN 0, above handle_message().

What to write. Define the two new functions of Listing 7 in `USER CODE BEGIN 0`, above `handle_message()`.

1. Build the reply functions.

(a) **`reply(reply_bytes, length_without_crc)`** appends a CRC and sends the reply. The array holds `length_without_crc` filled bytes, excluding the CRC; reserve two additional bytes. Calculate the CRC of the filled bytes and append it in the byte order of Appendix A of the booklet. Send all filled bytes and both CRC bytes.

(b) **`exception(function, code)`** builds and sends an exception reply. The arguments are the request's function code and an exception code from Table 12 of the booklet. Build the reply of Appendix A.3 of the booklet, then send it with `reply()`.

2. Extend `handle_message()`. Keep Task B1's reception, CRC and address checks, counters, and LD2 write policy. Add reads and exception replies:

(a) **Read registers.** Add Modbus function `0x03`. The request gives the first register address and register count. Build a reply with the requested values from `regs`, then send it with `reply()` (Table 11 and Figure 27 of the booklet).

(b) **Handle invalid requests.** For a request addressed to this board with a matching CRC, call `exception()` when:

- the function is not supported.
- a requested register does not exist, or a write is not to `REG_LD2`.
- a request value is not valid.

Choose the code from Table 12 of the booklet.

What to demonstrate and think about.

3. **Test reads, exceptions, and noise rejection.**
   (a) **Predict exception codes.** Give the codes for both tests in Appendix B. Think about why each test gets its code.
   (b) **Predict CRC mismatches.** From Exercise 6(c), predict how `bad` compares with the number of send attempts with a flipped bit. Think about why.
   (c) **Demonstrate reads.** Run `lab02_modbus.py` with no option. Each round must read the LD2 value that it wrote.
   (d) **Demonstrate exceptions.** Run with `--exceptions`. Compare each exception code with your prediction.
   (e) **Demonstrate noise rejection.** Reset the board to clear both counters. Run with `--errors 0.3`. Compare the last successful read's `bad` with the final `[PC] done` count `with a wrong bit`. Think about the cause of any difference: Appendix B says when each count is sampled.
   (f) **Think about the counters.** Why does `good` increase between consecutive rounds with no option? Why must the CRC be checked before the address when counting CRC mismatches?
   (g) **Calculate the total time.** From Exercise 5, add the 8-byte request, 15-byte reply, and 3.5 character times of idle after each. Compare with the master's 0.2 s limit. Think of one reason for the difference.

Task B3: Compare software and hardware CRC calculation

In this task, you compare the time needed to calculate each request's CRC in software, with Listing 5, and in hardware, with the chip's CRC unit. Measure both with the cycle counter (Figure 8). Keep all checks and replies from Task B2.

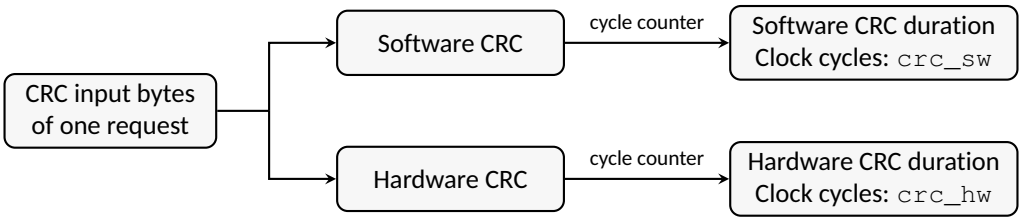


Figure 8: Task B3: The two CRC calculations and their measurements.

```
static void crc_unit_init(void)
{
}

static uint16_t crc_unit(const uint8_t *data, uint32_t byte_count)
{
}
```

Listing 8: Task B3: The new functions. Put them in USER CODE BEGIN 0, above crc_ok().

What to write. Define the two new functions of Listing 8 in `USER CODE BEGIN 0`, above `crc_ok()`.

1. Prepare the hardware CRC calculation.

(a) **`crc_unit_init()`** configures the CRC unit. Enable its clock and configure it for CRC-16/MODBUS. Table 7 of the booklet gives the standard's parameters; Table 9 gives the registers and clock enable bit. Call the function in `USER CODE BEGIN 2`.

(b) **`crc_unit(data, byte_count)`** calculates the CRC of `byte_count` input bytes in `data`. In this order:

(i) Reset the calculation to the standard's initial value.

(ii) Write each byte to `DR` with a byte access: `*(volatile uint8_t *)&CRC->DR = data[i]`.

(iii) Return the 16-bit result.

2. Time both calculations in `crc_ok()`. This function now checks the received CRC against both calculated results and records their durations. Use the same input bytes for both calculations, excluding the two received CRC bytes.

(a) **Measure the software CRC.** Read `DWT->CYCCNT` immediately before and after the complete `modbus_crc16()` call. Store the difference in `regs[REG_CRC_SW]`. The preparation started the counter (Section 7.2 of the booklet).

(b) **Measure the hardware CRC.** Read `DWT->CYCCNT` immediately before and after the complete `crc_unit()` call. Store the difference in `regs[REG_CRC_HW]`. This measurement includes the reset, input writes, and result read.

(c) **Check both results.** Return 1 only when the received CRC matches both results; otherwise, return 0.

What to demonstrate and explain.

3. **Predict, measure, and compare CRC durations.**
   (a) **Predict the speed ratio.** Find the number of CRC input bytes in an 8-byte request. Use Table 9 of the booklet to estimate software time divided by hardware time. State which calculation you expect to be faster.
   (b) **State your assumptions.** Give your assumed costs for software instructions and peripheral accesses. The CRC unit's listed time excludes reset, input writes, and result reads.
   (c) **Demonstrate CRC timing.** Run `lab02_modbus.py`. Each read must display `crc_sw` and `crc_hw` (Table 1). If no reply arrives, check the CRC unit settings.
   (d) **Compare the durations.** Convert both counts from one read to microseconds, using 170 MHz. Compare software time divided by hardware time with your prediction.
   (e) **Calculate time per byte.** Divide each duration by its number of CRC input bytes. Compare both results with the line's byte time from Exercise 5(a).

Demonstration and Sign-off

Keep these items, and be ready to demonstrate your work to the instructor at the test of this lab:

- Your answers to Pre-Laboratory Exercises 1 to 6.
- For each task, the items of Table 2. An image of the oscilloscope screen can be a photo or a screenshot.

Task	Copy of `main.c`	Oscilloscope image
A1	`lab2partAtask1.c`	No image.
A2	`lab2partAtask2.c`	PC4: the frame of one press of button B1.
A3	`lab2partAtask3.c`	PA5: the busy-wait delay.
B1	`lab2partBtask1.c`	No image.
B2	`lab2partBtask2.c`	No image.
B3	`lab2partBtask3.c`	No image.

Table 2: The items to keep for each task.

Appendix A: The Serial Monitor

The Serial Monitor is an extension of VS Code. It shows the characters that the board sends, and it sends the characters that you type.

Install the extension.

1. In VS Code, open the Extensions view (`Ctrl+Shift+X`).
2. In the search box, type `microsoft Serial Monitor`.
3. Select **Serial Monitor** from Microsoft (Figure 9). Click **Install**.

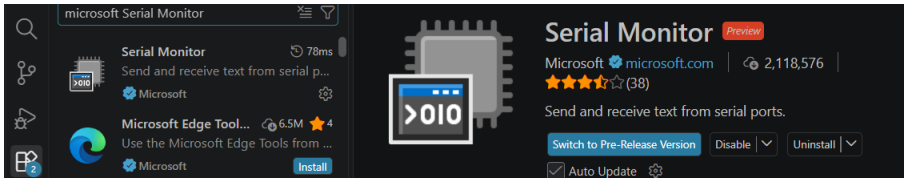


Figure 9: The Serial Monitor extension from Microsoft, in the Extensions view.

Open the tab.

1. In VS Code, select **Terminal**, then **New Terminal**.
2. In the bottom panel, click the **Serial Monitor** tab (Figure 10).

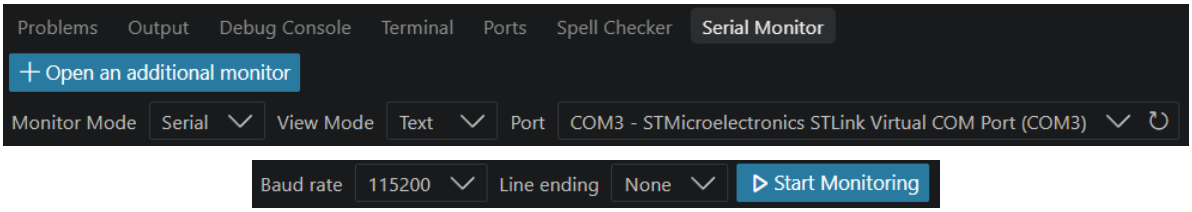


Figure 10: The Serial Monitor tab, set for the board. The second row continues the first row to the right.

Set the values.

- **Monitor Mode.** Keep **Serial**.
- **View Mode.** Keep **Text**.
- **Port.** Select *STMicroelectronics STLink Virtual COM Port*. Windows lists its COM number in Device Manager under **Ports (COM & LPT)**. If the port is absent, connect the board. Click the refresh button.
- **Baud rate.** Set the baud rate of your program.
- **Line ending.** Set **None**. Enter then sends the typed characters without additional line-end characters.

Start it. Click **Start Monitoring** (Figure 11):

- It shows each line that the board sends.
- The button changes to **Stop Monitoring**.

To send a line, type it in the message box at the bottom of the tab. Press Enter.

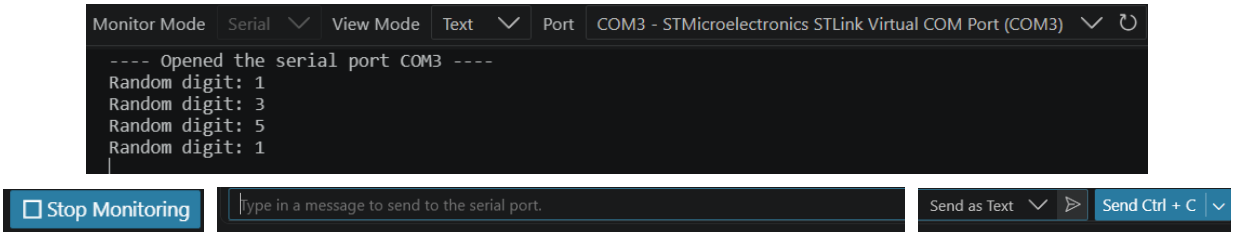


Figure 11: The received lines and the controls of the Serial Monitor.

Stop it. Click **Stop Monitoring** to close the port. Only one program at a time can open the port.

Appendix B: The Modbus master program

The course repository provides `lab02_modbus.py` in the `lab02-uart` folder. In a terminal, install its `pyserial` dependency:

```
python -m pip install pyserial
```

In each round, the program writes 1 or 0 to `REG_LD2`. Then it reads all five registers. It prints one line for each round, which starts with `[PC]`. With no reply in 0.2 s, it sends the request again. Each request has at most 5 send attempts; retries count as additional sends. The master also retries replies with an invalid CRC or an unexpected address, function, or length. A write reply must repeat the request. A read reply must contain the requested byte count.
The program exits with status 1 if any request fails. Exception replies are expected only with `--exceptions`.
The printed `good` and `bad` values are the board's message counters from Table 1, sampled when it handles the read request. The final `[PC] done` summary covers all send attempts, including any after the last successful read. Its `requests with no reply` count records requests that exhausted all 5 attempts without a usable reply.
With `--errors`, each send attempt can have one bit flipped. The summary's `with a wrong bit` count is the number of corrupted send attempts. The board's `bad` counter counts received messages with CRC mismatches. The summary's `invalid replies` counts replies rejected for a CRC mismatch, another slave address, or a response that does not match the pending request. It is separate from the board's `bad` counter.
With `--exceptions`, the two test requests read one register at address 9 and use unsupported Modbus function `0x04`. Each request has a matching CRC and targets this board.
Only one program can open the port of the board. Thus, stop the Serial Monitor first. Start the program in a terminal, in its folder:

```
python lab02_modbus.py                # 20 rounds: write, then read
python lab02_modbus.py --write-only   # write REG_LD2 only
python lab02_modbus.py --errors 0.3   # a wrong bit in 30 % of sends
python lab02_modbus.py --exceptions   # two requests to refuse
python lab02_modbus.py --rounds 100   # 100 rounds
python lab02_modbus.py --seed 7       # the same errors on each run
python lab02_modbus.py --port COM5    # if it cannot find the port
```